

HDOC 29

Name: Keshav Mittal SAP ID: 590032934

Batch: 16

1. Algorithm

1. Start main program.
2. Display the menu with options 1 to 4.
3. Read the user's choice.
4. Use a switch statement based on choice:
5. Case 1 (Count Stats):
 - Read array size n and validate $n > 0$.
 - Initialize counters $even=0$, $odd=0$, $pos=0$, $neg=0$, $zero=0$.
 - Loop ifrom 0 to $n-1$: Read $arr[i]$. If $arr[i] \% 2 == 0$, increment even, else increment odd. If $arr[i] > 0$, increment pos. If $arr[i] < 0$, increment neg. Else, increment zero.
 - Display all five counts.
6. Case 2 (Linear Search):
 - Read array size n and validate $n > 0$. Read the nelements into arr.
 - Read the key to search for. Initialize $found = 0$.
 - Loop ifrom 0 to $n-1$: If $arr[i] == key$, print index i , set $found = 1$, and break the loop.
 - If $found == 0$, display "Element not found".
7. Case 3 (Merge Arrays):
 - Read size $n1$ and elements for arr1. Read size $n2$ and elements for arr2.
 - Declare arr3 of size $n1 + n2$.
 - Loop ifrom 0 to $n1-1$: Copy $arr1[i]$ to $arr3[i]$.
 - Loop ifrom 0 to $n2-1$: Copy $arr2[i]$ to $arr3[n1 + i]$.
 - Display arr3.
8. Repeat steps 2-4 until choice == 4.
9. Stop

2. Test-Case Table

TC ID	Scenario Description	Inputs	Expected Output
TC-01	Count elements (Valid)	choice: 1, size: 5, Arr: 2 -3 0 4 -5	Even: 3, Odd: 2, Pos: 2, Neg: 2, Zero: 1
TC-02	Linear search (Found)	choice: 2, size: 3, Arr: 10 20 30, key: 20	Element 20 found at index 1.
TC-03	Linear search (Not Found)	choice: 2, size: 3, Arr: 10 20 30, key: 40	Element 40 not found in the array.
TC-04	Merge arrays (Valid)	choice: 3, size1: 2, Arr1: 1 2, size2: 2, Arr2: 3 4	Merged Array: 1 2 3 4
TC-05	Invalid dimension (Boundary)	choice: 1, size: -1	Error: Size must be positive.
TC-06	Exit program	choice: 4	Exiting program...

3. C Program Source Code

```
#include <stdio.h>

void countStats(void);
void linearSearch(void);
void mergeArrays(void);

int main() {
    int choice;
    do {
        printf("\n--- Array Operations Menu ---\n");
        printf("1. Count even, odd, positive, negative, and zero elements\n");
        printf("2. Search for an element using linear search\n");
        printf("3. Merge 2 arrays\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: countStats(); break;
            case 2: linearSearch(); break;
            case 3: mergeArrays(); break;
            case 4: printf("Exiting program...\n"); break;
            default: printf("Error: Invalid choice. Please enter 1-4.\n");
        }
    } while (choice != 4);

    return 0;
}

void countStats(void) {
    int n, i;
    int even = 0, odd = 0, pos = 0, neg = 0, zero = 0;

    printf("Enter array size: ");
    scanf("%d", &n);
    if (n <= 0) {
        printf("Error: Size must be positive.\n");
        return;
    }

    int arr[n];
    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);

        if (arr[i] % 2 == 0) even++;
        else odd++;

        if (arr[i] > 0) pos++;
        else if (arr[i] < 0) neg++;
        else zero++;
    }

    printf("Results:\n");
    printf("Even: %d, Odd: %d\n", even, odd);
    printf("Positive: %d, Negative: %d, Zero: %d\n", pos, neg, zero);
}

void linearSearch(void) {
    int n, i, key, found = 0;

    printf("Enter array size: ");
    scanf("%d", &n);
    if (n <= 0) {
        printf("Error: Size must be positive.\n");
        return;
    }

    int arr[n];
    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++) scanf("%d", &arr[i]);

    printf("Enter element to search for: ");
```

```

scanf("%d", &key);

for (i = 0; i < n; i++) {
    if (arr[i] == key) {
        printf("Element %d found at index %d.\n", key, i);
        found = 1;
        break;
    }
}

if (!found) {
    printf("Element %d not found in the array.\n", key);
}
}

void mergeArrays(void) {
    int n1, n2, i;

    printf("Enter size of first array: ");
    scanf("%d", &n1);
    if (n1 <= 0) {
        printf("Error: Size must be positive.\n");
        return;
    }
    int arr1[n1];
    printf("Enter %d elements for first array: ", n1);
    for (i = 0; i < n1; i++) scanf("%d", &arr1[i]);

    printf("Enter size of second array: ");
    scanf("%d", &n2);
    if (n2 <= 0) {
        printf("Error: Size must be positive.\n");
        return;
    }
    int arr2[n2];
    printf("Enter %d elements for second array: ", n2);
    for (i = 0; i < n2; i++) scanf("%d", &arr2[i]);

    int arr3[n1 + n2];
    for (i = 0; i < n1; i++) {
        arr3[i] = arr1[i];
    }
    for (i = 0; i < n2; i++) {
        arr3[n1 + i] = arr2[i];
    }

    printf("Merged Array: ");
    for (i = 0; i < n1 + n2; i++) {
        printf("%d ", arr3[i]);
    }
    printf("\n");
}

```

4. Compilation and Execution

```
(kali㉿kali)-[~/liak]
```

```
$ cc hdoc_29.c
```

```
(kali㉿kali)-[~/liak]
```

```
$ ./a.out
```

```
— Array Operations Menu —
```

1. Count even, odd, positive, negative, and zero elements
2. Search for an element using linear search
3. Merge 2 arrays
4. Exit

Enter your choice: 1

Enter array size: 5

Enter 5 elements: 2 5 2 0 -1

Results:

Even: 3, Odd: 2

Positive: 3, Negative: 1, Zero: 1

```
— Array Operations Menu —
```

1. Count even, odd, positive, negative, and zero elements
2. Search for an element using linear search
3. Merge 2 arrays
4. Exit

Enter your choice: 4

Exiting program...